

DISTRIBUTED PROCESSING

Distributed Processing, Defined

A distributed system is a collection of independent computers **that appears to its users as a single coherent system.**





Why We Build Systems This Way

1

Scalability

Add more machines to handle more users or data — instead of hitting the ceiling of one big machine

2

Fault Tolerance

If one node fails, others keep the system running — no single point of failure

3

Performance

Split work across nodes so tasks finish faster in parallel

4

Resource Sharing

Pool storage, compute, and data across an organization or the world





Distributed, Centralized, and Parallel Computing

Centralized

One machine, one memory space.
Simple but a single point of failure and
a hard scaling ceiling.

Parallel

Many processors, usually one shared
memory (or a tightly-coupled cluster).
Built for raw computation speed.

Distributed

Many independent machines, no
shared memory, connected by a
network. Built for scale, autonomy,
and availability.





Key Characteristics of Distributed Systems

1

Concurrency

Multiple nodes execute at the same time, coordinating shared resources

2

Scalability

Capacity grows by adding nodes, not upgrading one machine

3

Transparency

Complexity is hidden — the system looks unified to the user

4

Fault Tolerance

The system keeps working despite partial failures

5

Heterogeneity

Different hardware, OS, and networks can interoperate

6

No Global Clock

Nodes can't rely on perfectly synchronized time





Why Distributed Systems Are Hard

The "8 Fallacies of Distributed Computing" — assumptions that always break in practice:

1. The network is reliable

2. Latency is zero

3. Bandwidth is infinite

4. The network is secure

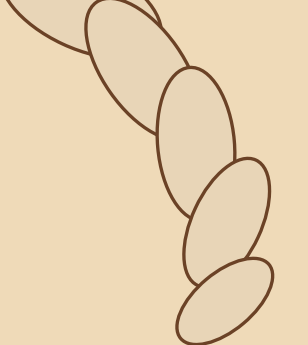
5. Topology doesn't change

6. There is one administrator

7. Transport cost is zero

8. The network is homogeneous





PART TWO

The Main Components

Nodes, networks, middleware, and the models that connect them



Nodes & Processes — the Basic Units





The Network — How Nodes Talk

1

Physical Layer

Cables, fiber, and wireless links carrying raw bits between machines

2

Protocols

TCP/IP, UDP — rules that structure and route messages reliably (or quickly)

3

Latency & Bandwidth

How long a message takes and how much data can flow — both are limited

4

Partitions

Networks can split or drop packets; the system must tolerate this



Middleware

Applications

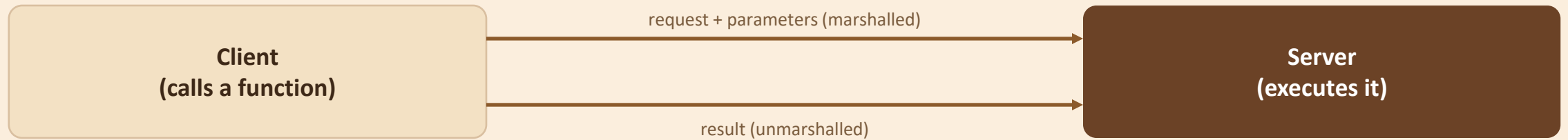
Middleware (RPC · message queues · object brokers · service discovery)

Operating Systems & Networks



Remote Procedure Calls (RPC)

The idea: call a function on another machine as if it were local.





Message Passing — the Underlying Model

Synchronous

Sender blocks until a response arrives — simple to reason about, but slower

Asynchronous

Sender continues immediately; response (if any) arrives later via a callback or queue

Point-to-Point

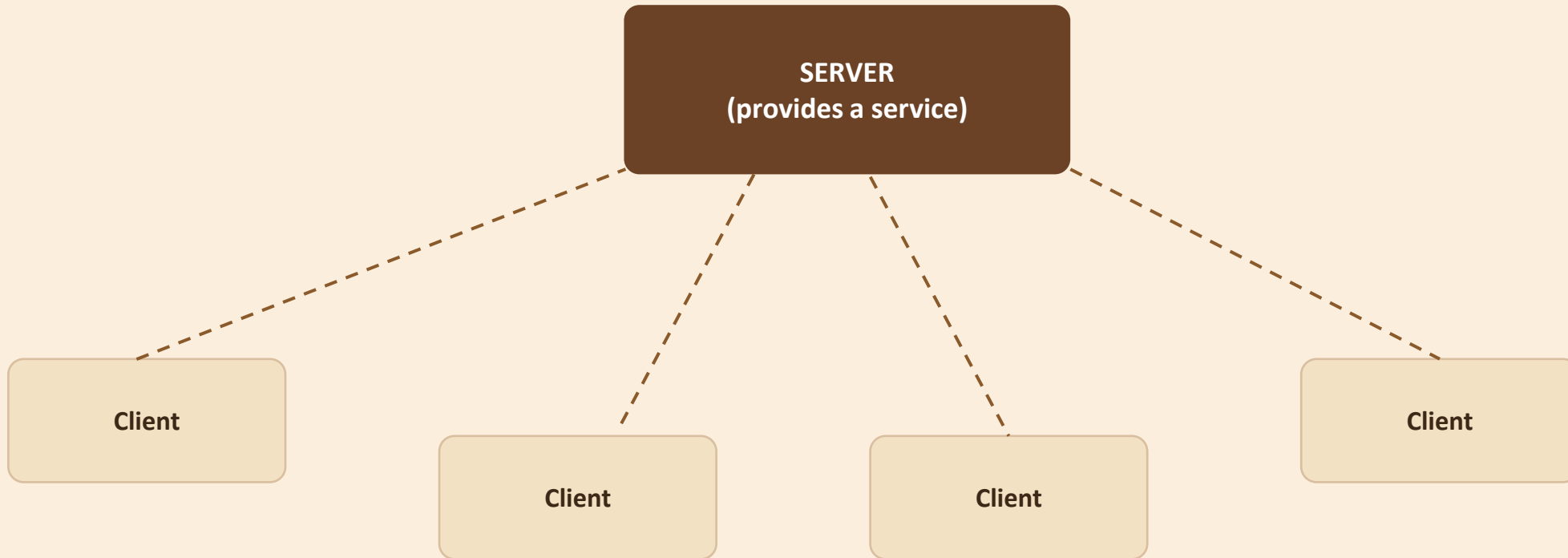
One sender, one receiver — direct and simple

Publish/Subscribe

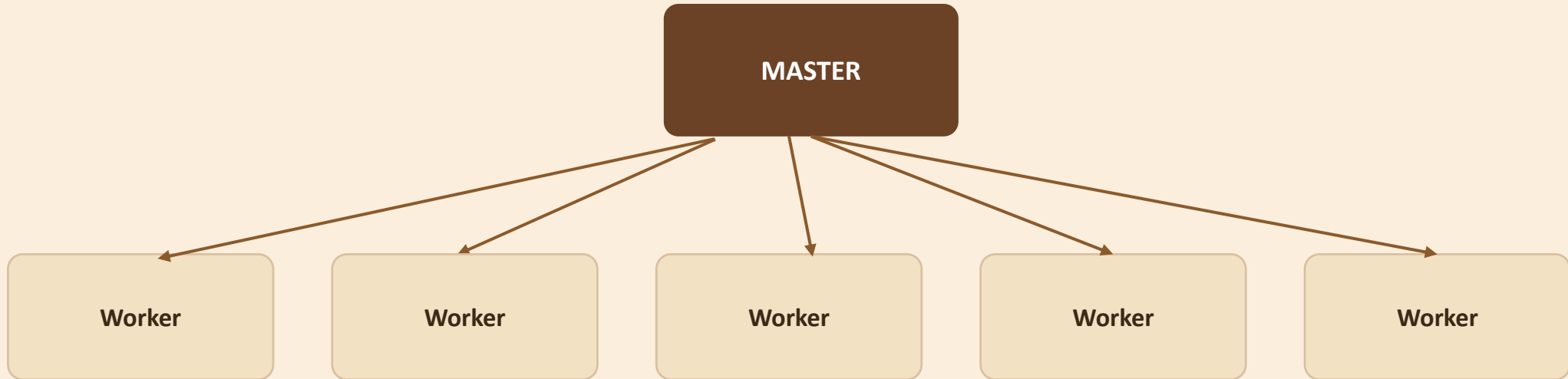
Senders publish to a topic; any number of subscribers receive it



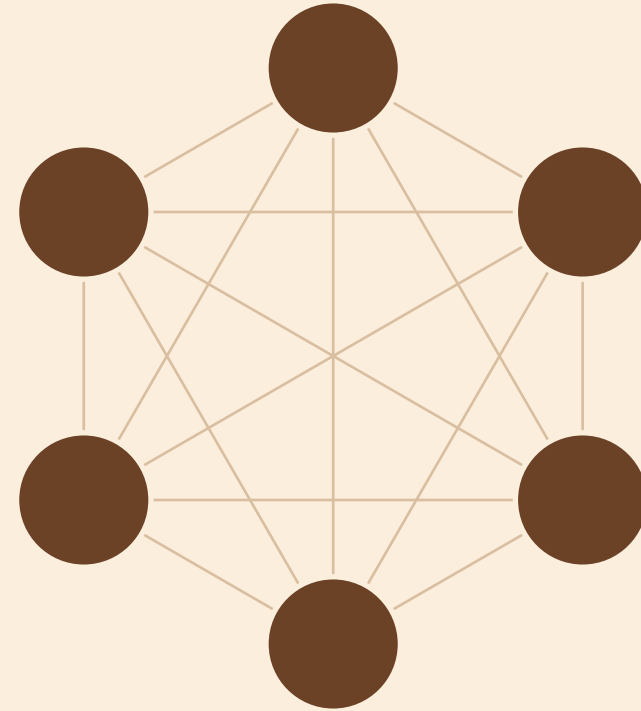
Client-Server Architecture



Master-Worker (Master-Slave) Architecture



Peer-to-Peer (P2P) Architecture

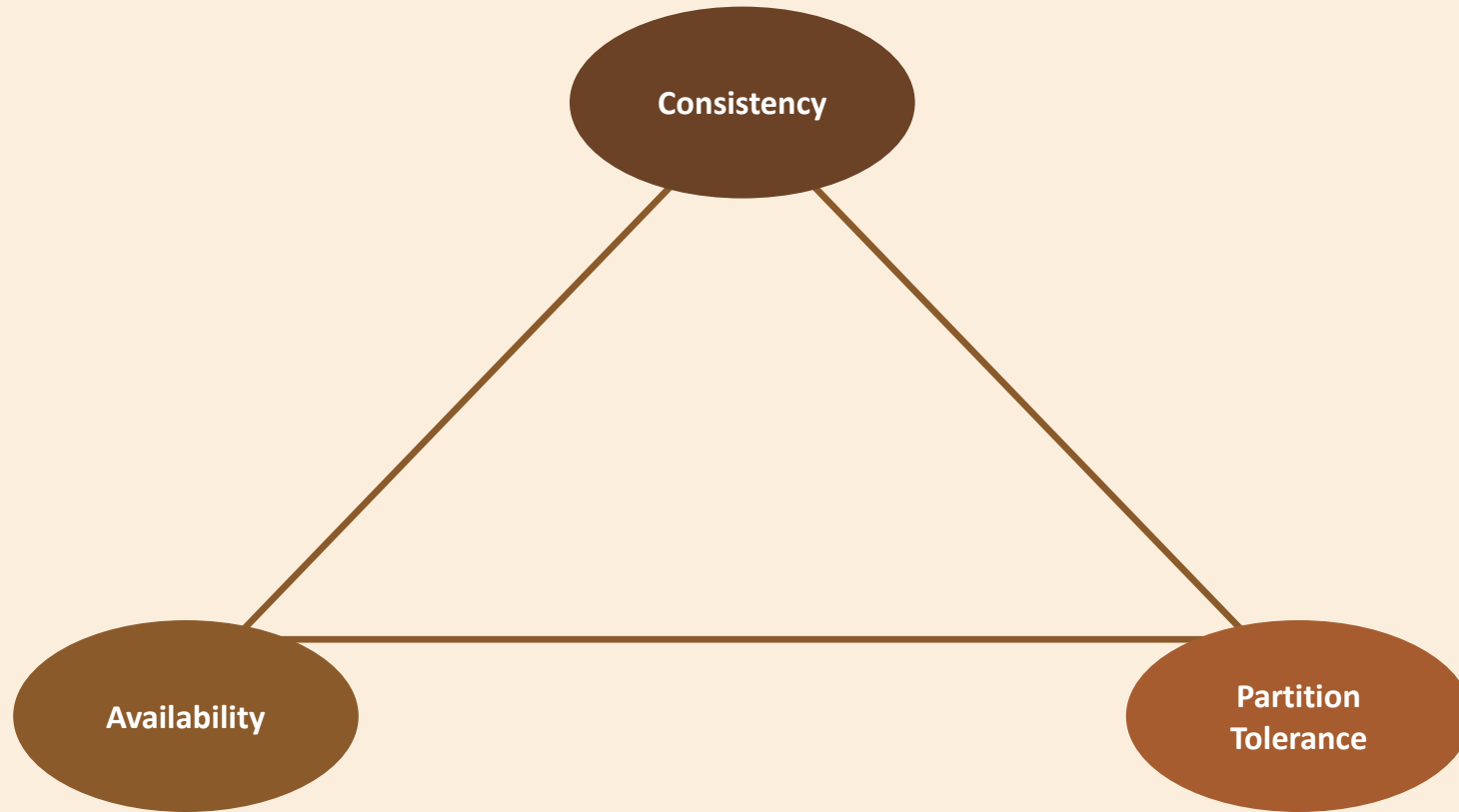


Feature	Peer-to-Peer (P2P)	Master-Slave
Control	No central controller	One master controls all slaves
Communication	Any node can communicate directly with any other node	Slaves usually communicate only with the master
Decision Making	Distributed among peers	Centralized at the master
Failure Impact	Failure of one node usually doesn't stop the network	If the master fails, the whole system may stop
Scalability	High; easy to add more peers	Limited by the master's capacity
Complexity	More complex synchronization	Simpler management
Examples	BitTorrent, Blockchain, P2P file sharing	SPI communication, I ² C (single master), USB (Host-Device)



The CAP Theorem

A distributed system can only fully guarantee two of the three at once:



Since networks do fail, partition tolerance is non-negotiable .



Fault Tolerance & Failure Handling

1

Redundancy

Run extra copies of data and services so one failure doesn't stop the system

2

Failure Detection

3

Failover

Traffic automatically shifts to a healthy replica when one node fails

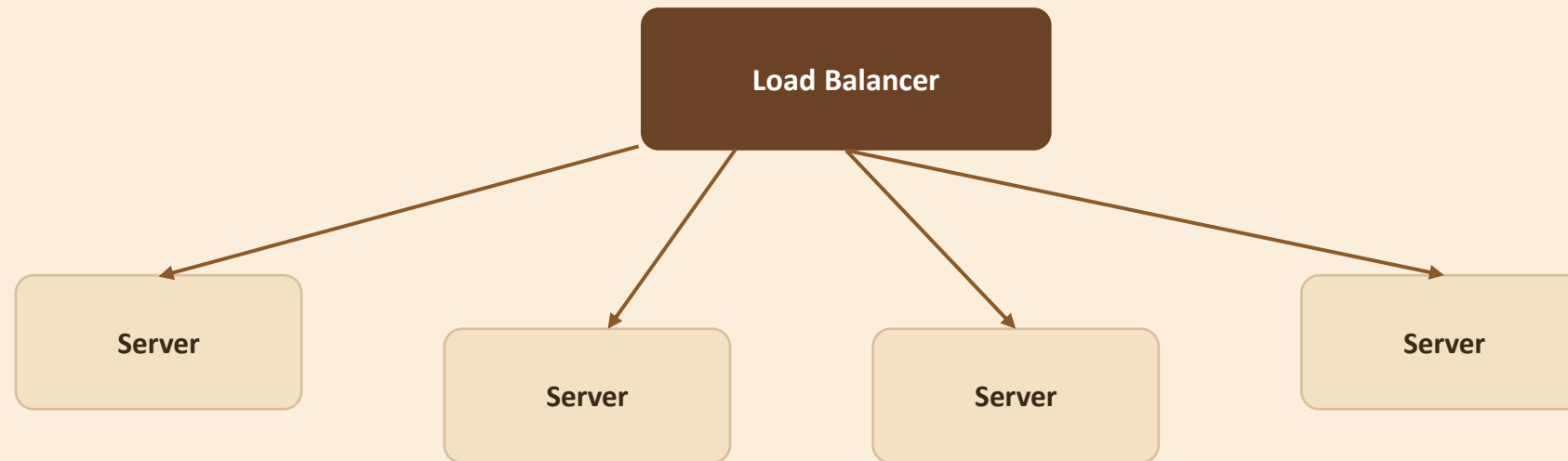
4

Checkpointing

Periodically save state so a restarted process can resume instead of starting over



Load Balancing





Frameworks That Put It All Together

1

MapReduce

The original programming model: map data across nodes in parallel, then reduce (combine) the results

2

Hadoop

Open-source ecosystem built around MapReduce and HDFS for batch processing at scale

3

Apache Spark

4

Kubernetes





Distributed Systems You Use Every Day

1

Streaming Video

Content is cached and served from data centers near you, coordinated globally

2

Ride-Hailing Apps

Location, matching, and payments processed across regional services in real time

3

Cloud Storage

Your files are chunked, replicated, and synced across multiple data centers

4

Online Banking

Transactions coordinate across banks and networks with strict consistency guarantees

5

Search Engines

Web-scale indexes are partitioned and queried in parallel across thousands of machines

6

Multiplayer Games

Player state is synchronized across servers and clients with low-latency messaging

